



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)

Y25 · 2026–2027 · Trimester 4 · Section 13 · Team 03

CAR RENTAL & MOBILITY PLATFORM

Review-3 · Methodology

Team Members

2520080037 – R. Rohit

2520090027 – G. Bhupathi

2520090082 – S. Anjani Shreyaj

Contents of Review-3



01
**Methodology
Overview**



02
Architecture and Flow



03
Novelty



04
Polyglot Data Design



05
**Cross-Service
Consistency**



06
Adaptability



07
Evaluation Plan



08
**Dissemination &
Access**



09
Summary & Next Steps

Proposed Methodology



Methodology Overview

We build the platform as a polyglot, service-oriented backend, where each workload uses the database and framework that suit it best.

Work moves in five phases, from data design to deployment and evaluation.

Every phase is checked through automated testing, image scanning, and deployment in GitHub Actions.

Methodology Phases



Design Data Models



Build Services



Connect Through Gateway



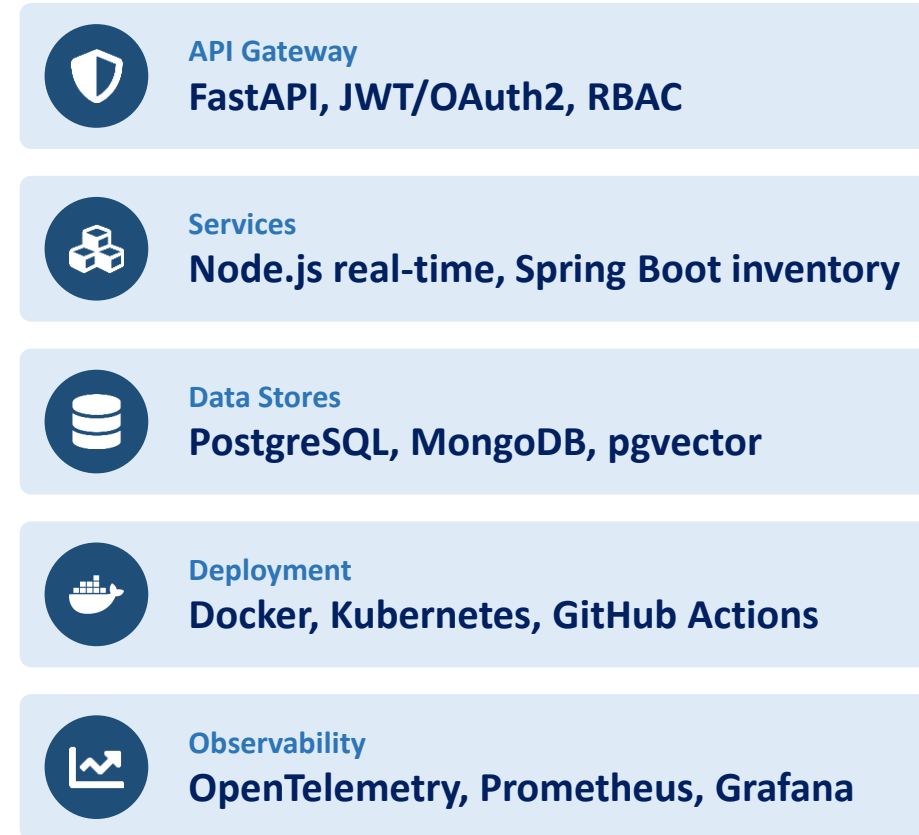
Containerize & Deploy



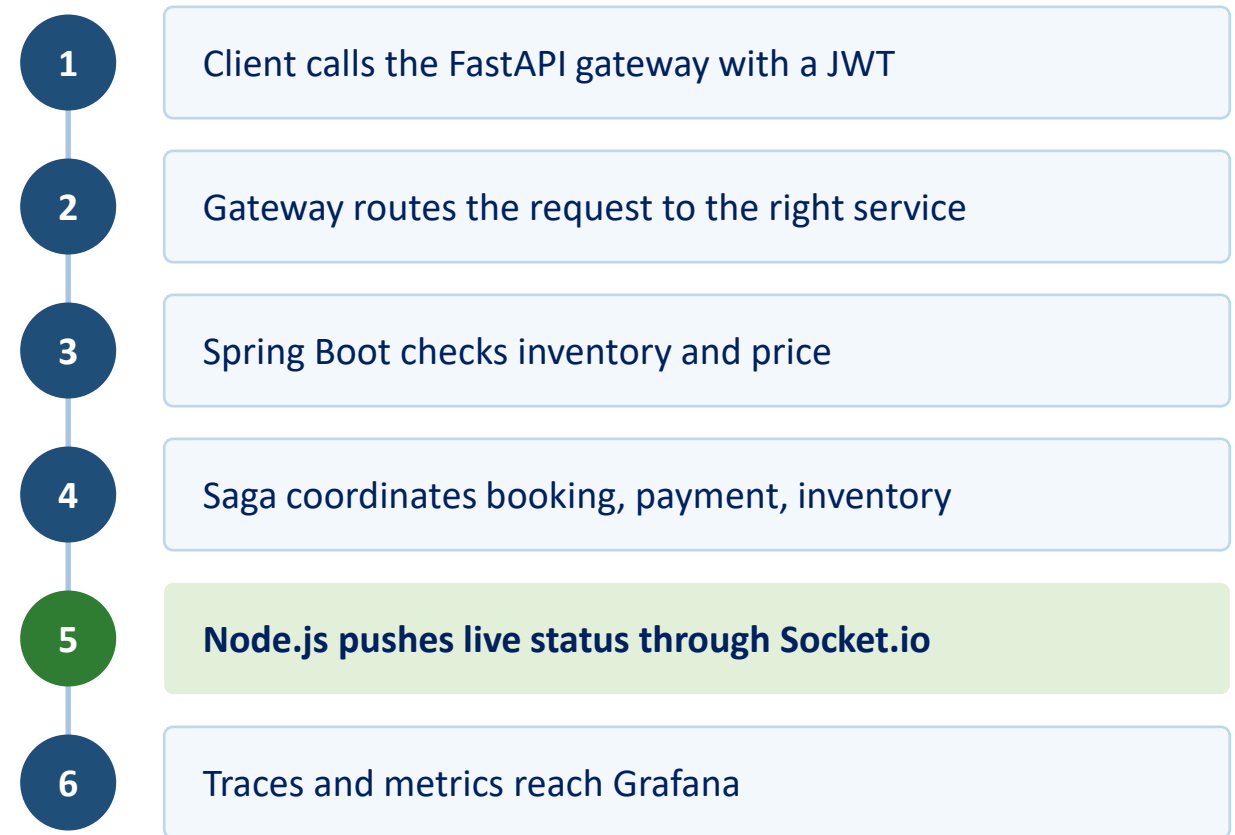
Observe & Evaluate

Architecture and Request Flow

Architecture Layers



Booking Request Flow



Novelty of the Methodology



Right Tool for Each Workload

Relational, document, and vector data in one service-oriented backend



Relational, document, and vector stores together



Saga with compensation instead of two-phase commit



HNSW embeddings for similar-vehicle search



Three frameworks behind one secure gateway

Polyglot Data Design



PostgreSQL

3NF schema, indexes, procedures, and triggers for bookings and payments.



MongoDB

Flexible BSON documents for listings, images, and reviews.



pgvector

Vehicle embeddings with an HNSW index and cosine similarity.



Redis + Socket.io

Fast live availability and tracking updates.

Cross-Service Consistency



A Saga with compensating transactions keeps booking, payment, and inventory consistent, without distributed two-phase commit.



Create Booking

Booking is saved as pending in PostgreSQL.



Reserve and Pay

Inventory holds the vehicle and payment is processed.



Confirm or Compensate

On any failure, compensating steps release the vehicle.

Adaptability of Methodology



**Services scale
on their own**



**Databases
chosen per
workload**



**New services
plug in via
gateway**



**Pattern fits
other booking
domains**



**Runs on
Docker or
Kubernetes**

Methodology Evaluation Plan

#	Aspect	What We Measure	Method / Tool
E1	Data integrity	No double bookings; correct rollback when a payment or reservation step fails	Automated tests of Saga failure cases
E2	API quality	Correct responses, input validation, and role-based access checks	Postman tests, OpenAPI contract
E3	Recommendations	Relevance of similar vehicles, and HNSW speed compared with exact search	Top-k result checks, query timing
E4	Scalability	Response time under load, and scaling of one service on its own	Load tests, Kubernetes scaling
E5	Observability	Traces, metrics, and dashboards across all services	OpenTelemetry, Prometheus, Grafana

Dissemination and Accessibility



Open Source Code

Code, README, and setup steps shared in a GitHub repository.



Documented APIs

Swagger / OpenAPI pages let anyone try the endpoints.



Easy Setup

Docker Compose starts the whole stack with one command.



Role-Based Access

Customers, agents, and admins each get suitable access.

Summary and Next Steps



The methodology is novel, adaptable, tested against clear checks, and shared openly through code and documentation.



What We Proposed

A polyglot, service-oriented backend with Saga-based consistency.



What We Will Evaluate

Data integrity, API quality, recommendations, and load behaviour.



Next Steps

Finish the services, run the tests, and prepare the final demo.